

토큰화

자연어 처리: 컴퓨터가 인간의 언어를 이해하고 해설 및 생성하기 위한 기술 의미

- 모호성(ambiguity): 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많다. 알고리즘은 이런 다양한 의미를 이해하고 명확하게 구분할 수 있어야 함.

- 가변성(Variability): 인간의 언어는 다양한 사투리, 강세, 신조어, 작문 스타일로 인해 매우 가변적. 알고리즘은 이런 가변성을 처리할 수 있어야 하며 사용중인 언어를 이해할 수 있어야 함.

- 구조(Structure): 문장이나 구의 의미를 이해할 때 구문을 파악하여 의미를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하며 의미를 추론하거나 분석할 수 있어야 한다.

=> 위와 같은 문제를 이해하고 구분할 수 있는 모델을 만드려면 말 뭉치(corpus)를 일정한 단위인 토큰으로 나눠야 함(Tokenization) <- Tokenizer사용

토큰라이저 구축 방법:

공백 분할: 공백 단위로 텍스트 분리, 개별 단어로 토큰화

정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.

어휘 사전 적용: 사전에 정의된 단어 집합을 토큰으로 사용. -> 직접 어휘 사전을 구축하기 때문에. 없는 단어나 토큰 존재 가능->Out of Vocab(OOV)

큰 어휘 사전 구축 시 차원의 저주(Curse of Dimensionality): 어휘 사전에 등장하는 토큰 개수만큼 차원 필요-> 벡터값이 거의 모두 0의 값을 가지는 희소 데이터로 표현됨.

머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝 적용.

단어 토큰화: 텍스트 데이터를 의미있는 단위인 단어로 분리하는 작업(띄어쓰기, 문장부호, 대소문자 등의 특정 구분자를 이용해 코트화 수행)

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

글자 토큰화: 띄어쓰기 뿐 아니라 글자 단위로 문장을 나누는 방법. 비교적 작은 단어 사전을 구축할 수 있는 장점.

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', ' ', '불', '가', '능', '한', ' ', ' ', 'c', 'g', '.', ' ', ' ', '시', '각', '적', ' ', ' ', '즐', '거', '움', '은', ' ', ' ', '최', '고', '!', ' ', ' ', '더', '더', '욱', ' ', ' ', 'o', 's', 't', '는', ' ', ' ', '더', '더', '욱', ' ', ' ', '최', '고', '!', '!', '']
```

형태소 토큰화: 텍스트를 형태소 단위로 나누는 토큰화 방법. 문법과 구조를 고려해 단어를 분리하고 이를 의미있는 단위로 분류하는 작업.

- 자립형태소
- 의존 형태소

형태소 어휘사전: 자연어 처리에서 사용되는 단어의 집합인 어휘사전 중에서도 각 단어의 형태소 정보를 포함하는 사전을 말한다. 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고있어, 형태소 분석 작업에서 매우 중요한 역할을 함.

텍스트 데이터를 형태소 분석하여 해당하는 품사(Part of speech, POS)를 태깅하는 작업을 품사태깅(POS Tagging)이라고 한다. 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있어 더욱 정확한 분석이 가능해진다.

KoNLPy: 한국어 자연어 처리를 위해 개발된 라이브러리. 명사추출, 형태소 분석, 품사 태깅 등의 기능 제공

잡바 언어 기반의 한국어 형태소 분석기로 JDK기반으로 개발됨.

임베딩:

토큰화 만으로는 모델학습 불가. 텍스트 자체를 이해할 수 없으므로 텍스트 벡터화(Text Vectorisation)과정이 필요.

텍스트 벡터화(Text Vectorisation): 텍스트를 숫자로 변환하는 과정.

-원 핫 인코딩(One-Hot Encoding) : 문서에 등장하는 단어를 고유한 색인 값으로 매핑-> 해당 색인 위치를 1로 표시하고 나머지는 0으로 표시

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

• I like apples: [1, 1, 1, 0]

• I like bananas: [1, 1, 0, 1]

-빈도 벡터화(Count Vectorization) : 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현

장점: 단어나 문장의 벡터 형태로 변환하기 쉽고 간단함. / 단점: 벡터의 희소성(Sparsity)이 크다는 단점-> 컴퓨팅 비용 문제와 차원 저주 문제, 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않으므로 두 문장이 의미적으로 유사하다고해도 벡터가 유사하게 나타나지 않을 수 있음.

=> 문제 해결: 워드 임베딩 기법

워드 임베딩 기법(Word Embedding): 단어를 고정된 길이의 실수 벡터로 표현하는 방법. 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론.

-동적 임베딩(Dynamic Embedding)

언어 모델:

입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델. 주어진 문장을 바탕으로 문맥을 이해하고 문장 구성에 대한 예측 수행.

주어진 문장 뒤에 나올 수 있는 문장은 매우 다양->완성된 문장 단위로 확률 계산 어려움.

=> 자기회귀 언어 모델: 문장 전체를 예측X, 하나의 토큰 단위로 예측.

Autoregressive Language Model(자기회귀 언어 모델): 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어 예측. 이전 단어 시퀀스 주어지면 -> 다음 단어의 확률 계산. -> 이전에 등장한 모든 토큰의 정보 고려.

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

-> 조건부 확률 이용

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

언어 모델에서 조건부 확률은 **연쇄법칙**(하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률을 이용해 계산하는 방식)을 이용해 계산.

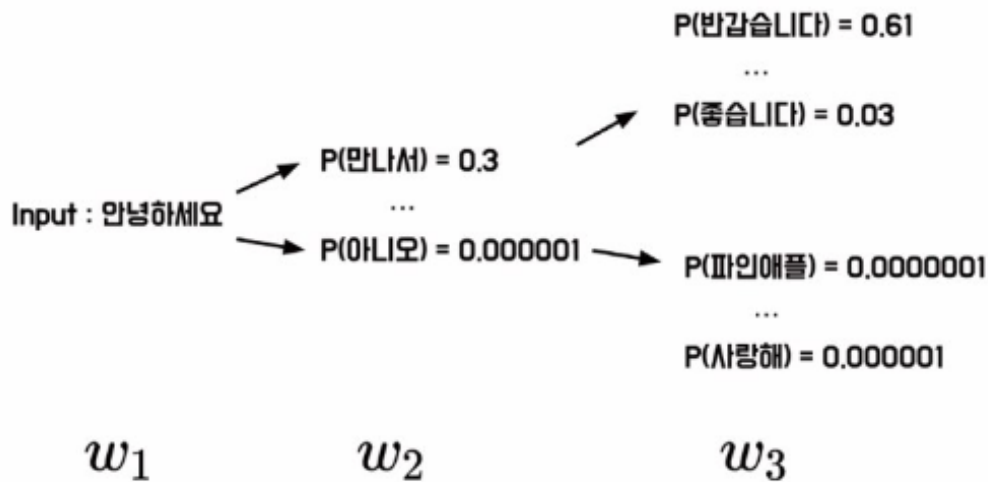


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

각 시점에서 다음에 올 토큰 예측이 중요: 입력 토큰 w_1 을 바탕으로 모델은 다음 토큰 w_2 가 등장할 확률 $P(w_1|w_2)$ 예측. -> $P(w_1|w_2, w_3)$ 예측

통계적 언어 모델(Autoregressive Language Model): 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어 예측

-**Markov Chain:** 빈도 기반의 조건부 확률 모델. 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태 예측.

(Ex. 말뭉치에 “안녕하세요” 등장 빈도:1000회, “안녕하세요 만나서” : 700회, “안녕하세요 반갑습니다”: 100회) 빈도 기반 조건부 확률:

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

단점)

1. 단어의 순서, 빈도에만 기초해 확률 예측-> 문맥을 제대로 파악하지 못하면 불완전/부적절한 결과 생성 가능.
2. 데이터 희소성(data sparsity)문제: 한번도 관측한 적 없는 데이터를 예측할 수 없음.

장점): 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률분포 생성-> 새로운 문장 생성 및 다양한 종류의 텍스트 데이터 학습 가능.

- 대규모 자연어. 데이터 처리에 효과적

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

가장 기초적인 통계적 언어 모델.

텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급->특정 단어 시퀀스가 등장할 확률 추론
입력 텍스트를 N개의 토큰으로 묶어서 분석(연속된 N개의 단어를 하나의 단어로 취급)

N=1 : unigram, N=2: Bigram, N=3:Trigram, N>=4: N-gram

Unigram : 안녕하세요 만나서 진심으로 반가워요

안녕하세요. 만나서. 진심으로. 반가워요

Bigram : 안녕하세요 만나서 진심으로 반가워요

안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요

Trigram : 안녕하세요 만나서 진심으로 반가워요

안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

모든 토큰 사용X, N-1개의 토큰만 고려해 확률 계산. =>

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

간단한 파이썬 코드 또는 NLTK라이브러리로 구현

TF-IDF(Term Ferquency-Inverse Document Frequency)

텍스트 문서에서 특정 단어의 중요도를 계산하는 방법, 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적인 가중치 의미. => TF-IDF는 BoW(Bag of Words)에 가중치를 부여하는 방법.

BoW(Bag of Words) : 문서나 문장을 단어의 집합으로 표현하는 방법. 원핫 인코딩과 다르게 등장 빈도 고려

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

-> 모든 단어는 동일한 가중치를 가짐 : 한번 등장하는 특정한 단어가 문장 분류에 중요한 역할을 하는경우 낮은 정확도

단어 빈도(Term Frequency, TF) : 문장 내에서 특정 단어의 빈도 수, 해당 단어의 상대적 중요도 측정.

TF높다고 중요한거는 아님. 자주 사용되는 단어는 전문 용어나 관용어로 간주 할 수 있음.

문서의 길이가 길어질수록 해당 단어의 TF값도 높아질 수 있음.

문서 빈도(Document Frequency, DF) : 한 단어가 얼마나 많은 문서에 나타나는지, 특정 단어가 많은 문서에서 나타날 경우, 문서 집합에서 단어가 나타나는 회수 계산.

DF값이 높으면 여러 문서에서 등장 하는 단어 -> 일반적으로 널리 사용되는 단어, 중요도가 낮을 수 있음

DF값이 낮으면 단어가 적은 수의 문서에서만 등장 -> 특정 맥락에서만 사용되는 단어->높은 중요도 가능

역문서 빈도(Inverse Document Frequency, IDF): $\log(\text{전체 문서 수} / \text{문서 빈도})$, 문서 내에서 특정 단어가 얼마나 중요한지

수식 6.7 역문서 빈도

높은 문서 빈도: 단어가 일반적 -> 낮은 중요도
문서 빈도의 역수 -> 단어 빈도수 낮음 -> 높은 IDF

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

TF-IDF: 문서 빈도 x 역문서 빈도

수식 6.8 TF-IDF

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

문서 내에 단어가 자주 등장 but, 전체 문서 내에 단어 적게 등장 -> TF-IDF 커짐
전체 문서에서 자주 등장할 확률이 높은 관사, 관용어 등 가중치 감소
scikit-learn 라이브러리로 계산